

Chapter 13

I Need to Make a Change, but I Don't Know What Tests to Write

I Need to
Make a
Change

When people talk about testing, they are usually referring to tests that they use to find bugs. Often these tests are manual tests. Writing automated tests to find bugs in legacy code often doesn't feel as efficient as just trying out the code. If you have some way of exercising legacy code manually, you can usually find bugs very quickly. The downside is that you have to do that manual work over and over again whenever you change the code. And, frankly, people just don't do that. Nearly every team I've worked with that depended on manual testing for its changes has ended far behind. The confidence of the team isn't what it could be.

No, finding bugs in legacy code usually isn't a problem. In terms of strategy, it can actually be misdirected effort. It is usually better to do something that helps your team start to write correct code consistently. The way to win is to concentrate effort on not putting bugs into code in the first place.

Automated tests are a very important tool, but not for bug finding—not directly, at least. In general, automated tests should specify a goal that we'd like to fulfill or attempt to preserve behavior that is already there. In the natural flow of development, tests that *specify* become tests that *preserve*. You will find bugs, but usually not the first time that a test is run. You find bugs in later runs when you change behavior that you didn't expect to.

Where does this leave us with legacy code? In legacy code, we might not have any tests for the changes we need to make, so there isn't any way to really verify that we're preserving behavior when we make changes. For this reason, the best approach we can take when we need to make changes is to bolster the area we want to change with tests to provide some kind of safety net. We'll find bugs

along the way, and we'll have to deal with them, but in most legacy code, if we make finding and fixing all of the bugs our goal, we'll never finish.

Characterization Tests

Okay, so we need tests—how do we write them? One way of approaching this is to find out what the software is supposed to do and then write tests based on those ideas. We can try to dig up old requirements documents and project memos, and just sit down and start writing tests. Well, that's one approach, but it isn't a very good one. In nearly every legacy system, what the system does is more important than what it is supposed to do. If we write tests based on our assumption of what the system is supposed to do, we're back to bug finding again. Bug finding is important, but our goal right now is to get tests in place that help us make changes more deterministically.

The tests that we need when we want to preserve behavior are what I call *characterization tests*. A *characterization test* is a test that characterizes the actual behavior of a piece of code. There's no "Well, it should do this" or "I think it does that." The tests document the actual current behavior of the system.

Here is a little algorithm for writing characterization tests:

1. Use a piece of code in a test harness.
2. Write an assertion that you know will fail.
3. Let the failure tell you what the behavior is.
4. Change the test so that it expects the behavior that the code produces.
5. Repeat.

In the following example, I'm reasonably sure that a `PageGenerator` is not going to generate the string "fred":

```
void testGenerator() {  
    PageGenerator generator = new PageGenerator();  
    assertEquals("fred", generator.generate());  
}
```

Run your test and let it fail. When it does, you have found out what the code actually does under that condition. For instance, in the preceding code, a freshly created `PageGenerator` generates an empty string when its `generate` method is called:

```
.F
Time: 0.01
There was 1 failure:
1) testGenerator(PageGeneratorTest)
   junit.framework.ComparisonFailure: expected:<fred> but was:<>
       at PageGeneratorTest.testGenerator
         (PageGeneratorTest.java:9)
       at sun.reflect.NativeMethodAccessorImpl.invoke0
         (Native Method)
       at sun.reflect.NativeMethodAccessorImpl.invoke
         (NativeMethodAccessorImpl.java:39)
       at sun.reflect.DelegatingMethodAccessorImpl.invoke
         (DelegatingMethodAccessorImpl.java:25)
```

FAILURES!!!

Tests run: 1, Failures: 1, Errors: 0

We can alter the test so that it passes:

```
void testGenerator() {
    PageGenerator generator = new PageGenerator();
    assertEquals("", generator.generate());
}
```

The test passes now. More than that, it documents one of the most basic facts about the `PageGenerator`: When we create one and immediately ask it to generate, it generates an empty string.

We can use the same trick to find out what its behavior would be when we feed it other data:

```
void testGenerator() {
    PageGenerator generator = new PageGenerator();
    generator.assoc(RowMappings.getRow(Page.BASE_ROW));
    assertEquals("fred", generator.generate());
}
```

In this case, the error message of the test harness tells us that the resultant string is “<node><carry>1.1 vectrai</carry></node>”, so we can make that string the expected value in the test:

```
void testGenerator() {
    PageGenerator generator = new PageGenerator();
    assertEquals("<node><carry>1.1 vectrai</carry></node>",
        generator.generate());
}
```

There is something fundamentally weird about doing this if you are used to thinking about these tests as, well, tests. If we are just putting the values that the software produces into the tests, are our tests really testing anything at all?

What if the software has a bug? The expected values that we're putting in our tests could just simply be wrong.

This problem goes away if we think of our tests in a different way. They aren't really tests written as a gold standard that the software must live up to. We aren't trying to find bugs right now. We are trying to put in a mechanism to find bugs later, bugs that show up as differences from the system's current behavior. When we adopt this perspective, our view of our tests is different: They don't have any moral authority; they just sit there documenting what pieces of the system really do. When we can see what the pieces do, we can use that knowledge along with our knowledge of what the system is supposed to do to make changes. Frankly, it's very important to have that knowledge of what the system actually does someplace. We can usually figure out what behavior we need to add by talking to other people or doing some calculations, but short of the tests, there is no other way to know what a system actually does except by "playing computer" in our minds, reading code and trying to reason through what the values will be at particular times. Some people do that faster than others, but regardless of how fast we can do it, it's pretty tedious and wasteful to have to do it over and over again.

Characterization tests record the actual behavior of a piece of code. If we find something unexpected when we write them, it pays to get some clarification. It could be a bug. That doesn't mean that we don't include the test in our test suite; instead, we should mark it as suspicious and find out what the effect would be of fixing it.

There is a lot more to writing characterization tests than what I've described so far. In the page generator example, it seemed like we were getting test values blindly by throwing values at the code and getting them back in the assertions. We can do that if we have a good sense of what the code is supposed to do. Some cases, such as not doing anything to an object and then seeing what its methods produce, are easy to think of and worth characterizing, but where do we go next? What is the total number of tests that we can apply to something such as the page generator? It's infinite. We could dedicate a good portion of our lives to writing case after case for this class. When do we stop? Is there any way of knowing which cases are more important than others?

The important thing to realize is that we aren't writing black-box tests here. We are allowed to look at the code we are characterizing. The code itself can give us ideas about what it does, and if we have questions, tests are an ideal way of asking them. The first step in characterizing is to get into a state of curiosity about the code's behavior. At that point, we just write tests until we are satisfied that we understand it. Does that cover everything in the code? It might not. But then we do the next step. We think about the changes that we want to

make in the code and try to figure out whether the tests that we have will sense any problems that we can cause. If they won't, we add more tests until we feel confidence that they will. If we can't feel that confidence, it's safer to consider changing the software in a different way. Maybe we can do a piece of what we were considering first.

The Method Use Rule

Before you use a method in a legacy system, check to see if there are tests for it. If there aren't, write them. When you do this consistently, you use tests as a medium of communication. People can look at them and get a sense of what they can and cannot expect from the method. The act of making a class testable in itself tends to increase code quality. People can find out what works and how; they can change it, correct bugs, and move forward.

Characterizing Classes

We have a class, and we want to figure out what to test. How do we do it? The first thing to do is to try to figure out what the class does at a high level. We can write tests for the simplest thing that we can imagine it doing and then let our curiosity guide us from there. Here are some heuristics that can help:

1. Look for tangled pieces of logic. If you don't understand an area of code, consider introducing a *sensing variable* (301) to characterize it. Use sensing variables to make sure you execute particular areas of the code.
2. As you discover the responsibilities of a class or method, stop to make a list of the things that can go wrong. See if you can formulate tests that trigger them.
3. Think about the inputs you are supplying under test. What happens at extreme values?
4. Should any conditions be true at all times during the lifetime of the class? Often these are called invariants. Attempt to write tests to verify them. Often you might have to refactor to discover these conditions. If you do, the refactorings often lead to new insight about how the code should be.

The tests that we write to characterize code are very important. They are the documentation of the system's actual behavior. Like any documentation that you write, you have to think about what will be important to the reader. Put yourself in the reader's shoes. What would you like to know about the class you are working with if you'd never seen it? In what order would you like the information? When you use the xUnit frameworks, tests are just methods in a file.

You can put them in an order that makes it easier for people to learn about the code they exercise. Start with some easy cases that show the main intent of the class, and then move into cases that highlight its idiosyncrasies. Make sure you document the important things that you discover as tests. When you get to making your changes, often you'll find that the tests you've written are very appropriate for the work you are about to do. Whether we think about it consciously or not, the change that we set out to make often guides our curiosity.

When You Find Bugs

When you characterize legacy code, you will find bugs throughout the entire process. All legacy code has bugs, usually in direct proportion to how little it is understood. What should you do when you find a bug?

The answer depends on the situation. If the system has never been deployed, the answer is simple: You should fix the bug. If the system has been deployed, you need to examine the possibility that someone is depending on that behavior, even though you see it as a bug. Often it takes a bit of analysis to figure out how to fix a bug without causing ripple effects.

My bias is toward fixing bugs as soon as they are found. When behavior is clearly in error, it should be fixed. If you suspect that some behavior is wrong, mark it in the test code as suspicious and then escalate it. Find out as quickly as you can whether it is a bug and how best to deal with it.

Targeted Testing

After we've written tests to understand a section of code, we have to look at the things that we want to change and see if our tests really cover them. Here is an example, a method on a Java class that computes the value of fuel in leased tanks:

```
public class FuelShare
{
    private long cost = 0;
    private double corpBase = 12.0;
    private ZonedHawthorneLease lease;
    ...
    public void addReading(int gallons, Date readingDate){
        if (lease.isMonthly()) {
            if (gallons < Lease.CORP_MIN)
                cost += corpBase;
            else
                cost += 1.2 * priceForGallons(gallons);
        }
        ...
    }
}
```

```

        lease.postReading(readingDate, gallons);
    }
    ...
}

```

We want to make a very direct change to the `FuelShare` class. We've already written some tests for it, so we are ready. Here is the change: We want to extract the top-level if-statement to a new method and then move that method to the `ZonedHawthorneLease` class. The `lease` variable in the code is an instance of that class.

**Targeted
Testing**

We can imagine what the code will look like after we refactor:

```

public class FuelShare
{
    public void addReading(int gallons, Date readingDate){
        cost += lease.computeValue(gallons,
                                   priceForGallons(gallons));
        ...
        lease.postReading(readingDate, gallons);
    }
    ...
}

public class ZonedHawthorneLease extends Lease
{
    public long computeValue(int gallons, long totalPrice) {
        long cost = 0;
        if (lease.isMonthly()) {
            if (gallons < Lease.CORP_MIN)
                cost += corpBase;
            else
                cost += 1.2 * totalPrice;
        }
        return cost;
    }
    ...
}

```

What kind of tests do we need to make sure that we do these refactorings correctly? One thing is certain: We know that we aren't going to be modifying this piece of logic at all:

```

    if (gallons < Lease.CORP_MIN)
        cost += corpBase;

```

Having a test in place to see how the value is computed below the `Lease.CORP_MIN` limit would be nice, but it is not strictly necessary. On the other hand, this else-statement in the original code is going to change:

```
else
    valueInCents += 1.2 * priceForGallons(gallons);
```

When that code moves over to the new method, it will become this:

```
else
    valueInCents += 1.2 * totalPrice;
```

That's a small change, but it is a change nonetheless. If we can make sure that the else-statement executes in one of our tests, we're better off. Let's look at the original method again:

```
public class FuelShare
{
    public void addReading(int gallons, Date readingDate){
        if (lease.isMonthly()) {
            if (gallons < CORP_MIN)
                cost += corpBase;
            else
                cost += 1.2 * priceForGallons(gallons);
        }
        ...
        lease.postReading(readingDate, gallons);
    }
    ...
}
```

If we are able to make a `FuelShare` with a monthly lease and we attempt to `addReading` for a number of gallons greater than `Lease.CORP_MIN`, we'll go through that leg of the else:

```
public void testValueForGallonsMoreThanCorpMin() {
    StandardLease lease = new StandardLease(Lease.MONTHLY);
    FuelShare share = new FuelShare(lease);

    share.addReading(FuelShare.CORP_MIN +1, new Date());
    assertEquals(12, share.getCost());
}
```

When you write a test for a branch, ask yourself whether there is any other way that the test could pass, aside from executing that branch. If you are not sure, use a *sensing variable* (301) or the debugger to find out whether the test is hitting it.

One important thing to figure out when you are characterizing branches such as this is whether the inputs that you provide have special behavior that could lead a test to succeed when it should fail. Here's an example. Suppose that the code used doubles instead of ints to represent money:

```

public class FuelShare
{
    private double cost = 0.0;
    ...
    public void addReading(int gallons, Date readingDate){
        if (lease.isMonthly()) {
            if (gallons < CORP_MIN)
                cost += corpBase;
            else
                cost += 1.2 * priceForGallons(gallons);
        }
        ...
        lease.postReading(readingDate, gallons);
    }
    ...
}

```

We could run into some serious trouble. And, no, I'm not referring to the fact that the application probably leaks fractional cents all over the place because of floating-point rounding errors. Unless we pick our inputs well, we could make a mistake when we extract a method and never know it. One possible mistake could happen if we extract a method and make one of its arguments an int rather than a double. In Java and many other languages, there is an automatic conversion from doubles to ints; the runtime just truncates the value. Unless we take care to devise inputs that will force us to see that error, we'll miss it.

Let's look at an example. What would be the effect on the previous code if the value of `Lease.CORP_MIN` is 10 and the value of `corpBase` is 12.0 when we run this test?

```

public void testValue () {
    StandardLease lease = new StandardLease(Lease.MONTHLY);
    FuelShare share = new FuelShare(lease);

    share.addReading(1, new Date());
    assertEquals(12, share.getCost());
}

```

Because 1 is less than 10, we just add 12.0 to the initial value of cost, which is 0. At the end of the calculation, the value of cost is 12.0. That is perfectly fine, but what if we extract the method like this and declare the value of cost as a long rather than a double?

```

public class ZonedHawthorneLease
{
    public long computeValue(int gallons, long totalPrice) {
        long cost = 0;
        if (lease.isMonthly()) {

```

```
        if (gallons < CORP_MIN)
            cost += corpBase;
        else
            cost += 1.2 * totalPrice;
    }
    return cost;
}
```

**Targeted
Testing**

That test that we wrote still passes, even though we are silently truncating the value of `cost` when we return it. A conversion from `double` to `int` is being executed, but it isn't really being fully exercised. It does the same thing that it would if there was no conversion, if we were just assigning an `int` to an `int`.

When we refactor, we generally have to check for two things: Does the behavior exist after the refactoring, and is it connected correctly?

Many characterization tests look like “sunny day” tests. They don't test many special conditions; they just verify that particular behaviors are present. From their presence, we can infer that refactorings that we've done to move or extract code have preserved behavior.

How can we handle this? There are a couple of general strategies. One is to manually calculate the expected values for a piece of code. At each conversion, we see whether there is a truncation issue. Another technique is to use a debugger and step through assignments so that we can see what conversions a particular set of inputs triggers. A third technique is to use *sensing variables* (301) to verify that a particular path is being covered and that the conversions are exercised.

The most valuable characterization tests exercise a specific path and exercise each conversion along the path.

There is a fourth option also. We can just decide to characterize a smaller chunk of code. If we have a refactoring tool that helps us extract methods safely, we can slice up the `computeValue` method and write tests for its pieces. Unfortunately, not all languages have refactoring tools—and at times, even the tools that are available don't extract methods the way that you wish they would.

Refactoring Tool Quirks

A good refactoring tool is invaluable, but often people who have these tools have to resort to refactoring by hand. Here is one common case. We have a class A with some code that we'd like to extract in its `b()` method:

```
public class A
{
    int x = 1;
    public void b() {
        int y = 0;
        int c = x + y;
    }
};
```

If we want to extract the `x + y` expression in method `b` and make a method called `add`, at least one tool on the market will extract it as `add(y)` rather than `add(x,y)`. Why? Because `x` is an instance variable and it is available to whatever methods we extract.

A Heuristic for Writing Characterization Tests

1. Write tests for the area where you will make your changes. Write as many cases as you feel you need to understand the behavior of the code.
2. After doing this, take a look at the specific things you are going to change, and attempt to write tests for those.
3. If you are attempting to extract or move some functionality, write tests that verify the existence and connection of those behaviors on a case-by-case basis. Verify that you are exercising the code that you are going to move and that it is connected properly. Exercise conversions.

This page intentionally left blank